WEEK 02 MODULE
Programming Practice

1. Fraud Transaction Detection (Linear Search)

que.1 **What is Searching?**

- Searching is the process of finding whether a particular element exists in a collection of data or not.

Types of Searching :-

i. Linear Search
ii. Binary Search

- **Linear Search** is a searching technique in which we check each element one by one from the beginning until the required element is found or the array ends.

Key Points ;-

- Works on unsorted as well as sorted arrays.
- Elements are checked sequentially (one after another).

Example

Array: 1201 1305 1450 1408 1502

Search: 1450

Checking:

1201 ✗
1305 ✗
1450 ✓

Problem Statement
A bank stores transaction IDs in the order they occur. Since the transactions are not sorted, the security team needs to check if a suspicious transaction ID exists.

1. Problem Analysis

The bank maintains transaction IDs in the order they occur. Since transactions are recorded chronologically, the IDs are not guaranteed to be sorted. The objective is to determine whether a suspicious transaction ID exists within the available transaction records.

The major challenge is that the data is unsorted. Because of this, techniques that require ordered data, such as Binary Search, cannot be applied directly. Therefore, every transaction may need to be examined until the target transaction ID is located.

2. Algorithm Selection Analysis

To search within an unsorted collection of transaction IDs, Linear Search is the most appropriate technique.

Reasons:

- Works efficiently on unsorted data.
- Does not require any preprocessing or sorting.
- Simple implementation.
- Suitable when records are stored in the order of occurrence and must be searched directly.

Since the transaction IDs are not arranged in ascending or descending order, Linear Search becomes the natural choice.

3. Core Idea / Logic

The algorithm scans the transaction list sequentially from the first element to the last.

For each transaction ID:

- Compare the current transaction ID with the suspicious transaction ID.
- If both values match, the fraud transaction is present in the records.
- If they do not match, continue searching the remaining transactions.
- If the entire list is traversed without a match, conclude that the transaction does not exist.

The search process terminates immediately once a match is found, avoiding unnecessary comparisons.

4. Execution Flow

i. Input Phase

- Read the number of transaction records.
- Store all transaction IDs in an array.
- Read the suspicious transaction ID.

Input Interpretation:

- The first input value represents the total number of transaction IDs/records (n).
- The second input line contains N transaction IDs which are stored in an array.
- The third input line contains the suspicious transaction ID that must be searched within the transaction records.

ii. Search Phase

- Start from index 0.
- Compare each array element with the suspicious ID.
- Continue until:
    - Match is found, or
    - Last element is checked.

iii. Output Phase

If a match exist: Fraud Transaction Found

Otherwise: Transaction Not Found

Step-by-Step Program Execution

Using the sample input:

5
1201 1305 1450 1408 1502
1450

Step 1

User enters 5.

int n = sc.nextInt();

n = 5

Step 2

Array of size 5 is created.

int arr[] = new int[n];

Step 3

User enters transaction IDs.

1201 1305 1450 1408 1502

Array becomes:

Index : 0 1 2 3 4
Value :1201 1305 1450 1408 1502

Step 4

User enters suspicious transaction ID.

int target = sc.nextInt();
target = 1450

Step 5

Initialize flag.

boolean found = false;

Step 6

Linear Search begins.

and then continue comparison by comparison.

5. Key Observations

- The array does not need to be sorted.
- Every element has an equal chance of being examined.
- Best-case scenario occurs when the suspicious ID is the first element.
- Worst-case scenario occurs when the suspicious ID is the last element or absent from the array.
- The algorithm may require checking all transaction records before reaching a conclusion.

6. Program Structure Analysis (Linear Search)

i. Variable n

int n;

Purpose:

- Stores the number of transaction IDs entered by the user.
- Determines the size of the array.
- Controls the number of iterations required for input and searching.

ii. Array arr[ ]

int arr[ ] = new int[n];

Purpose:

- Stores all transaction IDs.
- Provides a structured way to access each transaction using an index.
- Enables traversal during the Linear Search process.

iii. Variable target

int target;

Purpose:

- Stores the suspicious transaction ID entered by the user.
- Acts as the search key against which all transaction IDs are compared.

iv. Boolean Variable found

boolean found = false;

Purpose:

- Tracks whether the suspicious transaction ID has been located.
- Initially assumes the transaction is absent.
- Changes to true when a matching transaction ID is found.

v. Input Loop

```
for(int i = 0; i < n; i++){
   arr[ i ] = sc.nextInt();
}
```

Purpose:

- Accepts transaction IDs from the user.
- Stores each value into the array.
- Ensures all N records are captured before searching begins.

vi. Search Loop

for(int i = 0; i < n; i++)

Purpose:

- Traverses the array sequentially.
- Examines each transaction ID one at a time.
- Implements the Linear Search algorithm.

vii. Comparison Statement

if(arr[i] == target)

Purpose:

- Compares the current transaction ID with the suspicious transaction ID.
- Determines whether the required record has been found.

viii. Break Statement

break;

Purpose:

- Terminates the search immediately after finding a match.
- Prevents unnecessary comparisons.
- Improves efficiency when the target appears before the end of the array.

ix. Output Decision Structure

if(found)

Purpose:

- Determines which message should be displayed.
- Produces the final result based on the search outcome

7. Program:

```
package week2_module;
import java.util.Scanner;
public class FraudTransactionDetectionLinearSearch {

public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        // Input number of transaction IDs
        System.out.print("Enter the number of transaction IDs: ");
        int n = sc.nextInt();

        // Create array
        int arr[] = new int[n];

        // Input transaction IDs
        System.out.print("Enter the " + n + " transaction IDs: ");
        for(int i = 0; i < n; i++){
                arr[i] = sc.nextInt();
        }




        // Input suspicious transaction ID
```

```
        System.out.print("Enter the suspicious of transaction ID: ");
        int target = sc.nextInt();

        // Flag variable
        boolean found = false;

        // Linear Search
        for(int i = 0; i < n; i++){
                if(arr[i] == target){
                        found = true;
                        break;
                }
        }

        // Display result
        if(found){
                System.out.println("Fraud Transaction Found");
        }
        else{
                System.out.println("Transaction Not Found");
        }
        sc.close();
        }
}
```

8. Program's output

```
Enter the number of transaction IDs: 5
Enter the 5 transaction IDs: 1201
1305
1450
1408
1502
Enter the suspicious of transaction ID: 1450
Fraud Transaction Found
```

9. Screenshots :

- program :

FraudTransactionDetectionLinearSearch.java

```java
package week2_module;
import java.util.Scanner;
public class FraudTransactionDetectionLinearSearch {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter the number of transaction IDs: ");          // Input number of transaction IDs
           int n = sc.nextInt();
           int arr[] = new int[n];                                          // Create array
           System.out.print("Enter the " + n + " transaction IDs: ");       // Input transaction IDs
           for(int i = 0; i < n; i++){
               arr[i] = sc.nextInt();
           }
           System.out.print("Enter the suspicious of transaction ID: ");    // Input suspicious transaction ID
           int target = sc.nextInt();
           boolean found = false;                                           // Flag variable

           // Linear Search
           for(int i = 0; i < n; i++){
               if(arr[i] == target){
                   found = true;
                   break;
               }
           }
           // Display result
           if(found){
               System.out.println("Fraud Transaction Found");
           }
           else{
               System.out.println("Transaction Not Found");
           }

           sc.close();
       }
   }
```

- program's output :

```
Problems  Javadoc  Declaration  Console
<terminated> FraudTransactionDetectionLinearSearch [Java Application]
Enter the number of transaction IDs: 5
Enter the 5 transaction IDs: 1201
1305
1450
1408
1502
Enter the suspicious of transaction ID: 1450
Fraud Transaction Found
```